

Modul Praktikum Flutter Lanjutan

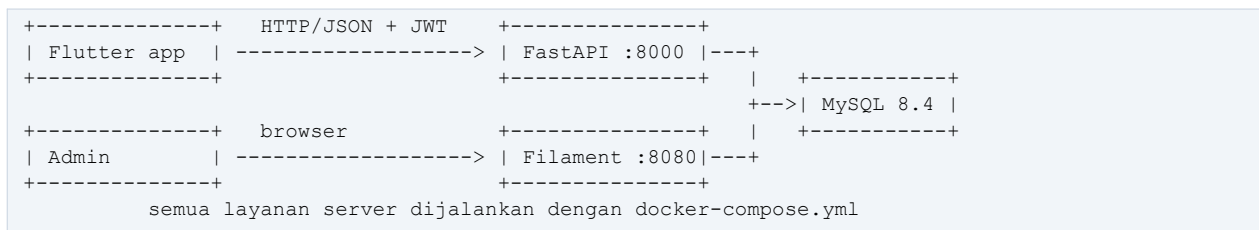
Pertemuan 8: Docker Compose dan Kerangka Backend TabungKu Cloud

Durasi	Level	Bentuk kerja	Prasyarat
150 menit	Lanjut	Individu	Pertemuan 1–7, terutama mini proyek TabungKu

1. Latar Belakang

Pada pertemuan 7, TabungKu menyimpan data di SQLite di dalam HP. Cara ini sederhana, tetapi data hilang bila HP rusak, tidak bisa dibuka dari HP lain, dan tidak bisa dipantau oleh pengelola. Aplikasi nyata biasanya terdiri atas tiga bagian: **aplikasi mobile**, **backend (API)**, dan **dashboard admin**, yang berbagi satu database di server.

Pada pertemuan 8–14, TabungKu dikembangkan menjadi **TabungKu Cloud**. Fitur, aturan bisnis, dan tampilannya tetap sama dengan pertemuan 7, sehingga fokus bisa diarahkan ke arsitektur:



Pert.	Topik	Hasil akhir pertemuan
8	Docker Compose, MySQL, kerangka FastAPI	<code>docker compose up</code> menjalankan MySQL + API; aplikasi menampilkan status server
9	FastAPI CRUD: SQLAlchemy, Alembic, Pydantic	Endpoint /kegiatan lengkap dan terdokumentasi di Swagger
10	Autentikasi JWT	Register/login; setiap pengguna hanya melihat datanya sendiri
11	Integrasi Flutter dengan API	UI TabungKu memakai data server lewat <i>repository pattern</i>
12	Dashboard Laravel + Filament	Admin mengelola kategori, pengguna, dan memantau kegiatan
13	Upload berkas dan pengujian	Foto bukti struk; pytest untuk backend
14	<i>Hardening</i> , deploy, proyek akhir	Compose produksi di belakang Nginx, backup, CI

2. Tujuan Pembelajaran

Setelah praktikum ini, mahasiswa mampu:

1. Menjelaskan perbedaan *image*, *container*, *volume*, dan *network* pada Docker.
2. Menulis `Dockerfile` untuk aplikasi Python dan `docker-compose.yml` berisi beberapa layanan.
3. Memakai variabel lingkungan (`.env`), *healthcheck*, dan `depends_on` agar layanan menyala berurutan.
4. Membuat API pertama dengan FastAPI dan membuka dokumentasinya di `/docs`.
5. Menghubungkan aplikasi Flutter ke API di laptop dari emulator maupun HP fisik.

3. Alat dan Bahan

- **Docker Desktop** (Windows/macOS) atau Docker Engine + plugin Compose (Linux). Alokasikan minimal 4 GB RAM untuk Docker.
- Flutter SDK, emulator Android atau HP fisik, dan editor (VS Code disarankan, dengan ekstensi Docker dan Python).
- Kode TabungKu dari pertemuan 7 (`pert7/praktikum7/lib/`).
- Koneksi internet untuk mengunduh image (`mysql:8.4`, `python:3.12-slim`) sekitar 700 MB.
Unduh sebelum kelas: `docker pull mysql:8.4` dan `docker pull python:3.12-slim`.
- Port yang dipakai: **8000** (API), **3307** (MySQL dari laptop), **8081** (Adminer, tugas).

4. Teori Singkat

Mengapa Docker? Tanpa Docker, setiap mahasiswa harus memasang MySQL, Python, dan PHP dengan versi yang sama, dan sering terjadi masalah "di laptop saya jalan". Docker membungkus aplikasi beserta seluruh kebutuhannya, sehingga cukup satu perintah untuk menjalankan semuanya dengan versi yang identik.

Istilah	Penjelasan	Analogi
Image	Cetakan aplikasi yang tidak berubah (OS minimal + runtime + kode)	Resep kue
Container	Image yang sedang berjalan; bisa dibuat dan dihapus kapan saja	Kue yang sudah jadi
Volume	Penyimpanan di luar container; data tetap ada walau container dihapus	Kulkas
Network	Jaringan virtual antar-container; antar-layanan saling memanggil dengan nama layanan (misalnya <code>mysql</code>)	Telepon antar-ruangan
Dockerfile	Langkah-langkah membuat image	Tulisan resep
<code>docker-compose.yml</code>	Daftar layanan beserta konfigurasinya, dijalankan sekaligus	Menu satu paket

FastAPI adalah framework Python untuk membuat REST API. Kelebihannya: cepat, validasi data otomatis dari *type hint*, dan dokumentasi interaktif (Swagger UI) yang dibuat otomatis di `/docs`.

Alamat server dari HP. `localhost` di emulator berarti emulator itu sendiri, bukan laptop. Gunakan alamat berikut:

Perangkat	Alamat API
Emulator Android	<code>http://10.0.2.2:8000</code> (alamat khusus menuju laptop)
Simulator iOS	<code>http://localhost:8000</code>
HP fisik (satu Wi-Fi dengan laptop)	<code>http://IP-LAPTOP:8000</code> , misalnya <code>http://192.168.1.10:8000</code>

Struktur folder. Kerjakan di folder baru `tabungku-cloud/` yang berisi `docker-compose.yml`, `.env.example`, `backend/` (FastAPI), dan `mobile/` (Flutter). Folder `admin/` (Laravel) ditambahkan pada pertemuan 12. Snapshot kode setiap pertemuan tersedia di repositori praktikum: `pertN/praktikumN/` dan `pertN/tugasN/`.

5. Langkah Praktikum

Bagian A: Menyiapkan Proyek

1. Pastikan Docker berjalan: `docker version` dan `docker compose version` menampilkan versi tanpa galat.
2. Buat struktur folder dan salin kode TabungKu pertemuan 7 ke folder `mobile/`:

```
mkdir tabungku-cloud && cd tabungku-cloud
mkdir -p backend/app
flutter create --project-name tabungku --platforms android,ios mobile
# salin isi pert7/praktikum7/lib/ ke mobile/lib/ (timpa main.dart)
```

3. Tambahkan paket `http` di `mobile/pubspec.yaml` (bersama `sqflite`, `path`, dan `shared_preferences` dari pertemuan 7), lalu jalankan `flutter pub get`.

Checkpoint A: `cd mobile && flutter run` menjalankan TabungKu persis seperti pertemuan 7.

Bagian B: API Pertama dengan FastAPI

Buat `backend/requirements.txt`. Versi ditulis tetap (`==`) agar semua mahasiswa memasang paket yang sama:

```
fastapi==0.142.2
uvicorn[standard]==0.54.0
pydantic-settings==2.15.0
SQLAlchemy==2.1.3
PyMySQL==1.2.3
cryptography==50.0.2
```

`cryptography` dibutuhkan `PyMySQL` untuk metode login bawaan MySQL 8. Buat `backend/app/__init__.py` (kosong), lalu `backend/app/config.py`. Konfigurasi dibaca dari *environment variable*, bukan ditulis di kode:

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    """Konfigurasi dibaca dari environment variable (diisi docker-compose)."""

    database_url: str = "mysql+pymysql://tabungku:rahasia_tabungku@localhost:3307/tabungku"

settings = Settings()
```

`backend/app/database.py`:

```
from sqlalchemy import create_engine

from .config import settings

# pool_pre_ping: cek koneksi sebelum dipakai, berguna bila MySQL sempat restart.
engine = create_engine(settings.database_url, pool_pre_ping=True)
```

`backend/app/main.py`:

```
from datetime import datetime

from fastapi import FastAPI

app = FastAPI(title="TabungKu API", version="0.1.0")

@app.get("/")
```

```
def beranda():
    return {"pesan": "Selamat datang di TabungKu API. Dokumentasi: /docs"}

@app.get("/health")
def health():
    """Dipakai aplikasi Flutter untuk memastikan server dapat dihubungi."""
    return {"status": "ok", "waktu_server": datetime.now().isoformat(timespec="seconds")}
```

Dekorator `@app.get("/health")` mendaftarkan fungsi sebagai endpoint GET `/health`. Nilai dict yang dikembalikan otomatis diubah menjadi JSON.

Bagian C: Dockerfile

backend/Dockerfile berisi langkah membuat image API:

```
FROM python:3.12-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /code

# Pasang dependensi lebih dulu agar layer ini di-cache selama
# requirements.txt tidak berubah.
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000", "--reload"]
```

Urutan baris penting. Docker menyimpan hasil setiap baris sebagai *layer* dan memakai ulang layer yang tidak berubah. Karena `requirements.txt` disalin dan dipasang sebelum kode, mengubah kode tidak memicu `pip install` ulang. `--host 0.0.0.0` wajib agar API dapat diakses dari luar container.

Tambahkan juga backend/.dockerignore berisi `__pycache__` dan `*.pyc` agar berkas sampah tidak ikut masuk image.

Bagian D: docker-compose.yml dan .env

Password **tidak boleh** ditulis langsung di `docker-compose.yml` yang ikut di-commit. Simpan di `.env.example` sebagai contoh:

```
# Salin menjadi .env lalu sesuaikan. Jangan commit berkas .env!
MYSQL_ROOT_PASSWORD=rahasia_root
MYSQL_DATABASE=tabungku
MYSQL_USER=tabungku
MYSQL_PASSWORD=rahasia_tabungku
```

Buat `.gitignore` berisi `.env`. Lalu tulis `docker-compose.yml`:

```
name: tabungku

services:
  mysql:
    image: mysql:8.4
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: ${MYSQL_DATABASE}
      MYSQL_USER: ${MYSQL_USER}
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
      TZ: Asia/Jakarta
    ports:
      - "3307:3306" # 3307 di laptop agar tidak bentrok dengan MySQL/XAMPP lokal
```

```

volumes:
  - mysql_data:/var/lib/mysql
healthcheck:
  test: ["CMD-SHELL", "mysqladmin ping -h 127.0.0.1 -uroot -p${MYSQL_ROOT_PASSWORD} --silent"]
  interval: 5s
  timeout: 5s
  retries: 20
  start_period: 20s

api:
  build: ./backend
  restart: unless-stopped
  environment:
    DATABASE_URL: mysql+pymysql://${MYSQL_USER}:${MYSQL_PASSWORD}@mysql:3306/${MYSQL_DATABASE}
    TZ: Asia/Jakarta
  ports:
    - "8000:8000"
  volumes:
    - ./backend/app:/code/app # kode di laptop langsung dipakai container (auto-reload)
  depends_on:
    mysql:
      condition: service_healthy

volumes:
  mysql_data:

```

Bagian	Fungsi
<code>\${MYSQL_USER}</code>	Diisi otomatis dari berkas <code>.env</code> di folder yang sama
<code>ports: "3307:3306"</code>	Port laptop 3307 diteruskan ke port 3306 di dalam container
<code>volumes: mysql_data</code>	Data MySQL disimpan di volume bernama, sehingga tidak hilang saat container dibuat ulang
<code>@mysql:3306</code>	Di dalam network Compose, nama layanan <code>mysql</code> menjadi nama host
<code>healthcheck</code>	Perintah yang menandai MySQL siap. <code>-h 127.0.0.1</code> memaksa koneksi TCP, karena saat inisialisasi pertama MySQL sementara hanya membuka socket
<code>depends_on ... service_healthy</code>	API baru dijalankan setelah MySQL benar-benar siap
<code>./backend/app:/code/app</code>	<i>Bind mount</i> : perubahan kode di laptop langsung terlihat oleh <code>uvicorn --reload</code>
<code>\$\$</code>	Tanda <code>\$</code> yang di- <i>escape</i> agar dibaca oleh shell di dalam container, bukan oleh Compose

Bagian E: Menjalankan dan Perintah Dasar

```

cp .env.example .env
docker compose up -d --build      # build image lalu jalankan di latar belakang
docker compose ps                 # status layanan (tunggu mysql "healthy")
docker compose logs -f api        # log API (Ctrl+C untuk keluar)

```

Buka `http://localhost:8000/health` dan `http://localhost:8000/docs` di browser. Ubah teks pesan di `main.py`, simpan, lalu muat ulang browser: perubahan langsung terlihat tanpa build ulang.

Perintah	Kegunaan
<code>docker compose exec api sh</code>	Masuk ke shell container API
<code>docker compose exec mysql mysql -utabungku -p tabungku</code>	Membuka klien MySQL
<code>docker compose restart api</code>	Menjalankan ulang satu layanan

Perintah	Kegunaan
<code>docker compose down</code>	Menghentikan dan menghapus container (data di volume tetap ada)
<code>docker compose down -v</code>	Sekaligus menghapus volume, sehingga seluruh data database hilang

Checkpoint E: `docker compose ps` menunjukkan mysql *healthy* dan api *Up*; `/health` membalas `{"status": "ok", ...}`; dan `/docs` menampilkan Swagger UI.

Bagian F: Flutter Memeriksa Status Server

1. Izinkan aplikasi Android mengakses jaringan dan HTTP biasa (non-HTTPS, khusus pengembangan).
Di `mobile/android/app/src/main/AndroidManifest.xml`:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <!-- Wajib untuk mengakses backend lewat jaringan. -->
  <uses-permission android:name="android.permission.INTERNET"/>
  <application
    android:label="tabungku"
    android:name="${applicationName}"
    android:usesCleartextTraffic="true"
    ...
  >
```

2. Buat `lib/config/api_config.dart`. Alamat dibaca dari `--dart-define` agar tidak perlu mengubah kode saat berganti perangkat:

```
/// Bawaan `10.0.2.2` adalah alamat khusus emulator Android untuk menuju
/// `localhost` milik laptop. Untuk HP fisik atau simulator iOS, ganti saat
/// menjalankan aplikasi, misalnya:
/// flutter run --dart-define=API_URL=http://192.168.1.10:8000
const apiUrl = String.fromEnvironment(
  'API_URL',
  defaultValue: 'http://10.0.2.2:8000',
);

/// Batas waktu tunggu setiap permintaan ke server.
const batasWaktu = Duration(seconds: 10);
```

3. Buat `lib/services/health_service.dart`. Parameter `client` opsional memungkinkan pengujian tanpa server:

```
class StatusServer {
  final String status;
  final String waktuServer;
  final Duration latensi;

  const StatusServer({
    required this.status,
    required this.waktuServer,
    required this.latensi,
  });
}

/// Memanggil GET /health dan mengukur lama responsnya.
Future<StatusServer> cekServer({http.Client? client}) async {
  final c = client ?? http.Client();
  final jam = Stopwatch()..start();
  try {
    final res = await c.get(Uri.parse('${apiUrl}/health')).timeout(batasWaktu);
    jam.stop();
    if (res.statusCode != 200) {
      throw Exception('Server membalas kode ${res.statusCode}');
    }
    final data = jsonDecode(res.body) as Map<String, dynamic>;
```

```

    return StatusServer(
      status: data['status'] as String,
      waktuServer: data['waktu_server'] as String,
      latensi: jam.elapsed,
    );
  } finally {
    if (client == null) c.close();
  }
}

```

4. Buat `lib/pages/status_server_page.dart`: sebuah `StatefulWidget` dengan `FutureBuilder<StatusServer>` (pola pertemuan 4) yang menampilkan ikon hijau/merah, alamat API, waktu server, latensi, dan tombol **Periksa lagi** yang mengganti `_future`. Daftarkan rute `'/status'` di `main.dart` dan tambahkan `ListTile` "Status server" di halaman Pengaturan yang memanggil `Navigator.pushNamed(context, '/status')`.

5. Jalankan dengan alamat yang sesuai perangkat:

```

flutter run                                     # emulator Android
flutter run --dart-define=API_URL=http://localhost:8000 # simulator iOS
flutter run --dart-define=API_URL=http://192.168.1.10:8000 # HP fisik

```

6. Uji tanpa server memakai `MockClient` dari paket `http`, di `test/health_service_test.dart`:

```

test('cekServer membaca status dan waktu server', () async {
  final client = MockClient((req) async {
    expect(req.url.path, '/health');
    return http.Response(
      jsonEncode({'status': 'ok', 'waktu_server': '2026-10-06T08:00:00'}),
      200,
    );
  });
  final s = await cekServer(client: client);
  expect(s.status, 'ok');
});

```

Checkpoint F: halaman Status Server menampilkan "Server terhubung" dan latensinya. Setelah `docker compose stop api` lalu **Periksa lagi**, halaman menampilkan pesan galat. `flutter test` lulus.

6. Latihan Mandiri

1. Tambahkan endpoint `GET /info` yang mengembalikan nama aplikasi, versi, dan zona waktu server. Lihat hasilnya di `/docs`.
2. Ubah port API di laptop menjadi 9000 tanpa mengubah port di dalam container. Bagian mana yang diubah, dan apa yang perlu disesuaikan di Flutter?
3. Jalankan `docker compose down`, lalu `up -d` lagi. Buat tabel uji lewat `docker compose exec mysql ...` sebelum `down`. Apakah tabelnya masih ada? Ulangi dengan `down -v`.
4. Jalankan `docker images` dan `docker system df`. Berapa ukuran image API? Coba ganti `python:3.12-slim` dengan `python:3.12`, lalu bandingkan ukurannya.

7. Tugas

Kembangkan hasil praktikum dengan ketentuan:

- Tambahkan layanan **Adminer** (image: `adminer:5`) di port **8081** agar isi database dapat dilihat dari browser. Server: `mysql`.
- `GET /health` memeriksa koneksi database dengan `SELECT VERSION()` dan membalas `{"status", "waktu_server", "database", "mysql_version"}`. Bila database tidak dapat

dihubungi, API membalas **kode 503** dengan "database": "error" dan pesan detail.

- Halaman Status Server di Flutter menampilkan status **API** dan **Database** secara terpisah. Kode 503 harus dibaca sebagai "API hidup, database mati", bukan dianggap galat jaringan.
- Alamat API dapat diubah dari halaman Pengaturan (dialog dengan validasi format URL) dan tersimpan dengan `shared_preferences`.
- Unit test dengan `MockClient` untuk tiga kasus: sehat, 503, dan kode lain.
- Kumpulkan: repositori/folder proyek (tanpa `.env`), tangkapan layar `docker compose ps`, Swagger `/docs`, Adminer, Status Server (sehat dan saat `docker compose stop mysql`), serta README singkat cara menjalankan.

8. Rubrik Penilaian

Komponen	Bobot
Bagian A–F berjalan (checkpoint)	30%
Latihan mandiri	15%
Tugas: Adminer, <code>/health</code> dengan cek database dan kode 503	25%
Tugas: halaman status API/database, alamat API tersimpan, unit test	20%
Kerapian: <code>.env</code> tidak di-commit, README jelas	10%

9. Pertanyaan Refleksi

1. Apa yang terjadi pada data MySQL bila container dihapus, dan mengapa? Kapan data benar-benar hilang?
2. Mengapa API memanggil database dengan host `mysql`, sedangkan dari laptop memakai `localhost:3307`?
3. Apa risikonya bila `depends_on` tidak memakai `condition: service_healthy`?
4. Mengapa emulator Android tidak dapat memakai `localhost` untuk menghubungi laptop?
5. Mengapa kode status 503 lebih tepat daripada 200 saat database mati?

10. Troubleshooting Umum

Masalah	Solusi
<code>port is already allocated</code>	Port dipakai aplikasi lain (XAMPP, MySQL lokal). Hentikan aplikasi itu atau ubah port kiri, misalnya " <code>3308:3306</code> "
<code>Cannot connect to the Docker daemon</code>	Docker Desktop belum berjalan; buka aplikasinya dan tunggu sampai statusnya <i>running</i>
<code>docker compose up --build</code> diam lama tanpa keluaran	Unduhan image sedang berjalan. Jalankan <code>docker pull mysql:8.4</code> terpisah untuk melihat kemajuannya. Bila tetap macet saat build, coba <code>COMPOSE_BAKE=false docker compose up -d --build</code>
API Can't connect to MySQL server on 'mysql'	MySQL belum siap atau sedang inisialisasi pertama. Pastikan <code>depends_on</code> memakai <code>service_healthy</code> dan <code>healthcheck</code> memakai <code>-h 127.0.0.1</code>
Perubahan <code>requirements.txt</code> tidak berpengaruh	Image perlu dibuat ulang: <code>docker compose up -d --build</code>
Flutter: <code>Connection refused</code> di emulator	Pakai <code>10.0.2.2</code> , bukan <code>localhost</code> . Pastikan <code>docker compose ps</code> menunjukkan API berjalan
Flutter: HP fisik tidak tersambung	HP dan laptop harus di Wi-Fi yang sama. Izinkan port 8000 di firewall laptop. Cek alamat dengan <code>ipconfig/ifconfig</code> , lalu coba buka <code>http://IP:8000/health</code> di browser HP
<code>Cleartext HTTP traffic not permitted</code>	Tambahkan <code>android:usesCleartextTraffic="true"</code> (khusus pengembangan; produksi wajib HTTPS)
Lupa password MySQL setelah mengubah <code>.env</code>	Variabel <code>MYSQL_*</code> hanya dipakai saat volume pertama kali dibuat. Untuk mengulang dari awal: <code>docker compose down -v</code>

11. Catatan untuk Pengajar

- Snapshot setiap pertemuan disusun bertingkat: praktikum8 → tugas8 → praktikum9 → ... → tugas14. Mahasiswa yang tertinggal dapat melanjutkan dari snapshot pertemuan sebelumnya.
- Semua snapshot memakai `name: tabungku` di Compose, sehingga volume `tabungku_mysql_data` dipakai bersama. Migrasi database (mulai pertemuan 9) bersifat bertingkat, sehingga berpindah ke snapshot yang lebih baru tetap aman. Berpindah ke snapshot yang **lebih lama** membutuhkan `docker compose down -v`.
- Minta mahasiswa mengunduh image sebelum kelas. Unduhan bersamaan dari satu jaringan kampus bisa sangat lambat.
- Kode contoh telah diuji dengan Docker 29, Flutter 3.47.5, dan Python 3.12 (dalam image).

12. Referensi

- Dokumentasi Docker Compose: docs.docker.com/compose
- FastAPI, *First Steps*: fastapi.tiangolo.com/tutorial/first-steps
- Image MySQL resmi: hub.docker.com/_/mysql
- Jaringan emulator Android: developer.android.com/studio/run/emulator-networking